Dependency-Preserving Context Compaction for Tool-Using Agents

OpenBot

***Abstract*—A tool-using agent builds a history of requests, calls, approvals, and results. A short sliding window can keep a result while losing the call that gives it meaning. A summary may explain the missing call but cannot replace its typed protocol record. This design and analysis paper studies compaction of working context within one task. It defines a boundary scan over typed message records and a protocol for safely installing its result. The scan keeps a recent tail and moves its start backward until every linked tool exchange stays together. We prove that, under stated key and message assumptions, it finds the smallest closed suffix containing that tail. We also give a budget condition, a failure trace, and limits on what this result means. Structural closure neither proves that a provider accepts the request nor that a summary preserves every user constraint. We propose tests that measure these outcomes separately, with controlled tool chains and repeated compaction. No model benchmark is reported. A finite artifact checks 7,737 boundary cases exhaustively and 1,200 seeded key-set traces against independent oracles. The full scan splits no key on these traces; a one-step repair splits keys on 92 traces. These are structural results for a stated generator, not measured gains in model task success.**

***Index Terms*—Context compaction, Tool agents, Dependency closure, Working context**

I. Introduction

An agent needs more than recent conversation lines. A tool result refers to a call. An approval refers to a request, which refers to the call through another identifier, or ID. A context limit can remove one of these linked records while keeping another. A provider may reject the resulting input. A model may also misread structurally valid input if its summary loses an earlier user constraint.

These are two distinct problems. The first concerns the shape of the retained record. The second concerns its meaning. This paper calls the first property **structural closure**: every message linked to a retained tool exchange is also retained. It calls the second property **semantic retention**: the model can still use the task facts and constraints that matter for later work. Structural closure can be checked from IDs. Semantic retention usually needs a task and an expected outcome.

We propose a method that retains system messages, summarizes an older prefix, and keeps a recent suffix in full. It follows keys for calls and approvals backward before choosing the prefix. A validation and commit protocol then installs the new context. We prove the boundary rule and define tests for the fallible summary model.

The scope is working context within one task. Stored history does not make an old constraint visible to the next model call. User profiles, memory shared across tasks, retrieval indexes, and learning from past tasks are outside the scope.

The contribution is a precise contract for a familiar engineering step: moving a summary boundary past linked tool records. We give its unique smallest suffix, an independent interval characterization, and a commit rule that does not lose concurrent input. We separate this structural result from size fit and from task success. The claim is not that keeping tool pairs or using a summary with a recent tail is new. The contribution is the explicit scope, proof, failure cases, and independently checked finite model. The model-based task benchmark has not been run.

II. Model and Scope of the Guarantee

A. Messages and links

Let $M = (m_0, ..., m_{n-1})$ be the ordered non-system messages in one working context. A message is atomic for this method: compaction retains the whole message or puts the whole message into the summary source. Its parts may include text, media, tool calls, tool results, approval requests, or approval responses. System messages form a separate sequence S and remain in full.

Each message has a finite set of keys $K(i)$. A call with ID c and a result for that call both have key `call:c`. An approval request for call c with approval ID a has both `call:c` and `approval:a`. The response to that approval has `approval:a`. Plain text and media have no keys in this model. The key names use separate namespaces, so a call ID and an approval ID with the same text do not collide.

A key links all of its occurrences. A message with two keys joins two linked sets. Thus an approval response can reach a call through the request, even though the response has no call ID. A message containing several calls can also join their sets. We use this relation to keep typed records together; it does not infer the meaning of natural language references such as "use the earlier result."

For a boundary b, the retained suffix is all messages with index at least b. It is closed if every occurrence of every key in the suffix is also in the suffix. Equivalently, if a retained message has key k, the first occurrence of k must have index at least b. Keeping a suffix also keeps intervening unlinked messages. This is stronger than keeping only the linked component of the newest message, but it preserves the order and local setting of recent work.

The required recent tail contains the last $\min(r, n)$ non-system messages. Its initial boundary is $b_0 = \max(0, n - r)$. Here r is a nonnegative integer chosen by the host. The objective is to find the largest closed boundary at most b_0. A larger boundary means fewer retained messages. This

objective concerns message count among contiguous suffixes. It is not an optimum over arbitrary message subsets or over summary quality.

B. Assumptions

The closure and minimality proofs need only a fixed finite sequence of finite key sets. They do not need calls to come before results, or even need the keys to describe real calls. To interpret closure as preserving a valid exchange, all required structural links must appear in K. IDs must refer to one exchange within the snapshot. Reusing an ID for unrelated calls would join them and can retain too much. Missing or false IDs defeat the intended relation. Source validation requires one call record per call ID and one request per approval ID. Each result refers to an earlier call; each approval request refers to an earlier call; each response refers to an earlier request. Order includes part order within a message. This model allows one final result per call. Pending calls may lack a result, but cannot enter a model continuation that requires completed local calls.

We also assume that moving system messages to the front is valid for the host's prompt form. The proposed construction preserves their content and relative order within S, but it does not preserve their positions among non-system messages. The theorem concerns only the non-system sequence. It does not establish that this change in placement has no effect on model behavior.

Closure preserves complete exchanges but cannot create missing results or approvals. Provider rules may also constrain roles, adjacency, or message counts. Those rules need a separate adapter check.

III. Related Work

MemGPT manages a limited active context through several memory tiers [1]. ReSum uses repeated summaries for long search tasks and studies training for reasoning from those summaries [2]. AgentFold learns when and how much of a web agent's history to fold [3]. These works are closer comparisons than text compression alone. They address what state to keep and how an agent uses it. Our deterministic rule instead fixes which typed messages must stay together for a chosen recent tail. It can constrain a summary method, but we have not shown that adding it improves any of those systems.

Existing tools also preserve related records. Microsoft Agent Framework groups tool calls and their results into atomic units [4]. Cloudflare Agents moves compaction boundaries to avoid splitting call/result pairs and keeps source messages in storage [5]. Thus neither pair preservation nor a separate stored history is new here. Our all-key contract covers chains through approval IDs and messages that join several exchanges. A comparison with these tools must use their actual released code and adapters, rather than treating a plain sliding window as their behavior.

Prompt compression often asks which text can be removed while preserving task performance. LLMLingua uses budget control and token-level compression to shorten prompts [6]. Its concern with dependencies between compressed content is relevant, but our dependency is explicit: two records share a typed ID. Removing a word from a result and removing its parent call have different protocol effects. A text compressor may be used inside the older prefix after the structural boundary is fixed.

LongLLMLingua makes compression sensitive to the question and to the placement of key information in long input [7]. This suggests a useful way to choose what goes into a summary. It does not replace the need to keep all parts of a retained tool exchange. In an agent task, the next question may also be an action whose requirements are not known when the prefix is compressed. A summary tuned only to the current request can lose a constraint needed later.

Liu and colleagues study how relevant information's position affects long-context use [8]. Their tasks show why fitting input within a published size limit does not establish that the model uses it well. Our evaluation therefore varies the location of constraints before compaction and measures later use. We make no claim that a closed suffix removes position effects.

ReAct combines reasoning traces and actions within a task [9]. Its action and observation sequence motivates the need to preserve relationships across turns. Toolformer studies how models can learn which APIs to call and how to use their results [10]. Both concern tool use. Our method addresses the host's later choice of which typed records to place in a bounded request. It neither changes tool learning nor proves that the selected action is correct.

RULER tests long-context behavior with controlled length and task complexity, including tracing and aggregation [11]. We adopt that design principle for a proposed agent benchmark: control chain span, branching, and irrelevant material independently. Our benchmark also exercises approvals and multiple compactions. A retrieval score does not establish those outcomes. None of the cited model results is a result for this proposal.

Two recent studies make the distinction between structure and meaning concrete. Governance Decay reports loss of action constraints after summary or eviction and tests keeping those constraints outside lossy compaction [12]. TRACE compares continuations from the same task state with full and compacted context, and finds limited evidence for deciding when compaction will hurt from prior actions alone [13]. We cite their released preprints and do not treat their model results as a guarantee. They support two choices in our planned study: grade actual later actions, and use paired continuations from the same state.

IV. Dependency-Preserving Compaction

A. Boundary selection

The first pass records the earliest message for every key. The map has value semantics: setting a key replaces its prior value, and lookup returns that value. Scanning from newest to oldest makes the final value the smallest index. Duplicate occurrences of a key within one message do not change it. Every later lookup is defined because this pass visits the same key sets used by the boundary scan. The second pass starts at the newest message and moves backward. When the scan reaches a message in the retained region, it moves the

boundary to include the earliest occurrence of each key in that message. A move may add more messages. The continuing backward scan visits those messages and follows their keys too. No repeated scan over the whole history is needed.

```
Input: messages M, recent count r
first := empty map
for i from n - 1 down to 0:
  for key in keys(M[i]):
    first[key] := i
b := max(0, n - r)
for i from n - 1 down to 0:
  if i >= b:
    for key in keys(M[i]):
      b := min(b, first[key])
if b = 0:
  return Unchanged
source := M[0:b]
tail := M[b:n]
text := summarize(source, tools = none)
reject empty or length-cut text
candidate := systems + summary(text) + tail
check candidate size and source snapshot
commit candidate, leaving history intact
```

Listing 1. Boundary scan and summary flow. Map assignment replaces a value. Snapshot checks and context replacement form one atomic commit.

The algorithm uses the earliest occurrence rather than a call-only table. This matters for approvals. A response first points to its request through the approval key; the request then points to the call through the call key. It also matters when one message joins more than one exchange. A one-step lookup for the original tail would miss links introduced by the expanded region.

At boundary zero, preparation returns the unchanged prompt without a summary call. The caller still counts it and rejects a missed target. No removable prefix means compaction cannot reduce size under this policy.

B. Summary construction and acceptance

Preparation is a pure function: it reads a fixed input and returns a boundary and a summary request without making a model call. It rejects a negative or noninteger recent count and a nonpositive or noninteger token target. The request encodes the older prefix as JSON data. A defined free-text baseline asks for the user's goal, constraints, decisions, important facts, tool results, unfinished work, and any earlier summary. It asks the model to return only a short summary and not continue the task. No tools are available to the summary call.

The accepted summary becomes a labeled user message after the preserved system messages and before the retained tail. The tail is copied exactly, including media and typed tool parts. The summary text does not create a tool result, grant approval, or change a stored permission. Empty text and a response cut short by the output limit are rejected. These rules exclude two clear failure forms; they do not prove completeness or truth.

Define the context state as a revision number and the full set of values that determine model input: the window, instructions, tool schemas, model choice, and their formatting rules. Preparation must read that revision and all of those values from one consistent snapshot. An atomic snapshot read meets this rule. A version check before and after the reads also works when every write follows the same version rule and a changed version causes a retry. A preparation ticket keeps the snapshot revision, source values, boundary, and target. Summary generation occurs outside the context transaction. The same ticket can hold a saved summary response, but saving it does not publish a new working context.

Validation first checks typed records and the provider adapter's layout rules. It then counts the entire assembled candidate, including instructions and tools. If the count exceeds the target, the candidate fails. A count is a hard bound only if the selected counter is exact or a valid upper bound for the complete provider request. Otherwise size fit is an estimate and must be reported as such.

Commit is one atomic compare-and-replace operation. It succeeds only when the current context revision equals the ticket revision. On success it installs the candidate and increases that revision. Every writer must change model-input values and increase the same revision atomically. Revision values must not be reused while a ticket can still be accepted. It must read the current state inside that operation or compare its read revision and retry on conflict. Unrelated metadata need not change this revision. A failed comparison leaves the newer context in place and requires fresh preparation. The append-only task history is separate and is never rewritten by compaction.

Commit guarantee. Assume that preparation reads one consistent snapshot, all context writes use the atomic operation above, revisions are not reused, and the ticket records every model-input value read by preparation. Then a successful compaction cannot overwrite an append committed after its snapshot.

Proof. The atomic writes have an order. If the append comes first, it changes the revision, and the old ticket's comparison fails. If compaction comes first, the append must read the installed context or fail its own comparison and retry. The append cannot both commit after the snapshot and then vanish through acceptance of that ticket. QED.

This is a safety result, not a progress guarantee. A busy chat may prevent every summary from being installed. A writer outside the revision rule defeats the proof, as does accepting an old revision after it is reused. Reading an old window and then a newer revision without a snapshot check can also erase input: the new revision would pass even though the source window is old. A failed commit does not undo the summary call, refund its cost, or establish summary correctness.

V. Analytical Results and Worked Cases

A. Closure and minimality

Proposition 1: closure. After the backward scan, the retained suffix is closed under all keys in K.

Proof. The boundary never increases. Consider any final retained message m_i. If it was already retained when the scan reached i, the scan processed all of its keys. If the boundary moved left to include it, that move occurred while processing a larger index, so the scan had not yet passed i. It therefore also processed that message. Processing key k makes the boundary no greater than its first occurrence. Later moves can only lower the boundary. Every occurrence of k is at or after

its first occurrence, so all are retained. This holds for every key in every retained message. Following a chain of shared keys therefore never leaves the suffix. QED.

Proposition 2: minimality among suffixes. The returned suffix contains no more messages than any other closed suffix that contains the initial recent tail.

Proof. Let c be the boundary of any such suffix. Initially c is at most b_0. Maintain the claim that c is at most the algorithm's current boundary. When a retained message at index i is processed, i is at least that boundary, and hence at least c. The candidate suffix starting at c must contain this message. Since it is closed, it must also contain the first occurrence of each of the message's keys. Thus c is at most each value used to lower the boundary. The claim survives every update. At the end, Proposition 1 gives a closed suffix, and no other allowed closed suffix has a larger boundary. QED.

Minimality is relative to contiguous suffixes. Separate message groups could be shorter but would require another contract for omitted local context. The boundary can move to the first message, leaving nothing to summarize.

B. An independent interval characterization

For each key k present in the snapshot, let f_k and l_k be its first and last message indices. A boundary lies between messages; its domain is the integers from zero through n. Define the forbidden cuts of that key as the integers in $(f_k, l_k]$.

Proposition 3: exact cut test. A suffix starting at b is closed if and only if no key satisfies $f_k < b \leq l_k$.

Proof. Such a key has an occurrence before the cut and another in the suffix, so the suffix is not closed. Conversely, if closure fails, some retained key has an occurrence before the cut. Its first index is below b and its last index is at least b. It therefore forbids that cut. QED.

A key with one occurrence forbids no cut. Transitive links need no separate interval rule: a cut through any link in their chain already fails this test. The allowed cuts need not form one interval. For a key at indices 1 and 3 in a five-message sequence, cuts 0, 1, 4, and 5 are allowed; cuts 2 and 3 are not. Testing a nearby cut alone is therefore insufficient.

Let $C(b)$ be the greatest allowed cut at most b. Propositions 1 and 2 show that the scan returns $C(b_0)$. This gives three testable laws. It never removes part of the required tail, since $C(b) \leq b$. It is idempotent, which means $C(C(b)) = C(b)$. It is monotone: if $a \leq b$, then $C(a) \leq C(b)$, since every candidate cut for a is also a candidate for b. In particular, asking for more recent messages cannot produce a shorter retained suffix. These statements hold for arbitrary key sets and provide checks independent of any provider layout.

The interval characterization is a second oracle. Enumerate all candidate cuts, discard each cut covered by an interval, and select the greatest remaining cut. It does not use the scan's boundary updates. Agreement between these two implementations supports implementation correctness on the tested inputs; the proofs, rather than those samples, establish the general finite result.

C. A sliding-window counterexample

Define the sliding-window baseline to choose $b = b_0$ without reading any keys. It either drops the older prefix or replaces it with free text; both forms keep the same typed tail. Define a one-step repair to take the minimum of b_0 and the first occurrence of every key found in that original tail. It does not inspect messages added by the repair. Table 1 is an illustrative abstract trace for these exact rules. It uses canonical message parts; a provider may need another layout. The call IDs are `a` and `b`, and the approval ID is `p`. The original recent tail is messages 4 and 5.

TABLE I
ILLUSTRATIVE LINKED EXCHANGES ACROSS A PROPOSED CUT. INDEX 3 JOINS TWO EXCHANGES IN ONE ATOMIC MESSAGE.

Index	Message parts	Keys
0	User goal and constraint	None
1	Call a	call:a
2	Call b; request approval p for b	call:b, approval:p
3	Result a; approval response p	call:a, approval:p
4	Result b	call:b
5	User asks to continue	None

A plain two-message window keeps result `b` but loses its call and approval. A prose summary saying that the call was allowed does not restore the typed records. Starting at boundary 4, the backward scan sees `call:b` and moves to 2. It then reaches message 3, which is now retained. Its `call:a` key moves the boundary to 1. The final suffix is messages 1 through 5. Message 0 alone becomes the summary source. A repair that only follows keys in the original tail would stop at 2 and lose call `a` while keeping its result in message 3.

This trace also marks the limit of the result. Suppose message 0 said to use only a test account. If the summary says merely that the user wants an export, the suffix is structurally closed but a later action may violate the user's constraint. The missing information is plain text with no interaction key. The closure proof says nothing about it.

D. Time, space, and budget

Let L be the total number of key occurrences, and let q be the number of distinct keys. The algorithm visits each message twice and each key occurrence at most twice. With constant expected map access, it takes expected $O(n + L)$ time and $O(q)$ map space. A map with logarithmic operations gives $O(n + L \log(q + 1))$. Immutable map updates do not justify an unconditional constant-time claim; their allocation and sharing costs depend on the map implementation. Cumulative allocation may exceed live map space. These bounds assume bounded key length; otherwise key extraction and hashing add their own costs. Summary generation, JSON encoding, and token counting have separate costs proportional to their input or model work.

For budget reasoning, use an additive charge model. Let F be fixed input cost, including preserved system text, instructions, and tool schemas. Let $T(b)$ be retained-tail cost, H be

the summary wrapper, and s be summary cost. Let B be the allowed input budget after any output reserve. A candidate fits when

$$F + T(b) + H + s \leq B. \tag{1}$$

If a useful summary must have cost at least $s_{\min}$, a necessary condition is $F + T(b) + H + s_{\min} \leq B$. A summary of that size gives **size fit** under this model, but may not preserve all needed facts.

Corollary 4: least retained charge. If each whole message has a fixed nonnegative charge, the scan's suffix has no greater retained charge than any other closed suffix containing the same recent tail. Every other allowed suffix contains this one by Proposition 2, so its charge is this charge plus nonnegative charges for extra messages. Therefore, if the scan's suffix alone exceeds the available budget, no other closed suffix under this policy can fit. This conclusion holds for fixed material and a fixed minimum summary charge. It does not optimize the total request when summary length depends on the cut, or when token charges change under serialization. Exact provider tokenization may not be additive across text boundaries, so the final serialized request still needs its actual counter.

As an explicitly illustrative calculation, take $B = 8000$, $F = 1200$, $T(b) = 6200$, and $H = 50$. The summary allowance is $8000 - 1200 - 6200 - 50 = 550$ tokens. A 700-token summary misses the target by 150. If the budget instead is 7000, the fixed material costs 7450. Even an empty summary cannot fit under these charges. Removing a linked call would break the structural contract. A host must report this limit or choose a different authorized policy, such as a larger model window or a smaller required tail.

The summary request has its own input limit. A small final candidate does not imply that the prefix, its JSON form, and summary instructions fit the summary model. Before generation, the protocol checks the summary input limit and fixed retained cost. It supplies the derived summary allowance, then counts the complete candidate after generation. As an ablation, define a total-target prompt that supplies only B and asks the model to leave room for recent messages. That instruction does not enforce the derived allowance. Both versions must still count the assembled candidate before commit.

VI. Semantic Retention Has a Separate Contract

A summary is a lossy representation. "Lossy" means that some source information is not kept exactly. The loss can be harmless when it removes repeated status text. It can be harmful when it removes a prohibition, changes a number, merges alternatives, or reports an unfinished action as complete. Structural validation cannot detect these changes because the summary is plain text.

There is also a quantitative limit. Fix a summary vocabulary of size $V \geq 2$ and a maximum summary length of s tokens. There are at most $Q = \sum_{j=0}^{s} V^j$ possible summaries. Suppose the removed prefix encodes an arbitrary vector of d independent binary constraints. Keep the tail, model state, and all outside inputs identical across vectors.

Proposition 5: finite-summary limit. If $2^d > Q$, a deterministic summary and continuation cannot answer every later constraint query correctly for every such prefix without reading the removed information.

Proof. There are 2^d input vectors and at most Q summaries. Two different vectors therefore have the same summary. They differ at some position j. A later query asking for that constraint gives the continuation the same input in both cases, but requires different answers. At least one answer must be wrong. QED.

For a randomized method with no shared state beyond the summary and a worst-case zero-error guarantee, every summary that can be emitted with positive probability must identify the vector exactly. Their output supports must therefore be disjoint, which gives the same bound. A permitted error rate requires a different analysis. This result assumes no retrieval, saved hidden state, or other channel carrying the removed bits. It does not predict loss on ordinary tasks: their constraints may be repeated or compressible. It rules out a universal guarantee under the stated finite channel, not the value of summaries in practice.

A proposed structured summary lists the goal, active constraints, completed actions with result references, unresolved choices, and next work. It keeps source message IDs, marks uncertainty, and never treats old permission as current permission. These fields extend the free-text baseline but do not prove completeness.

Repeated compaction includes earlier summaries in later summary input. A fact omitted once is unavailable to later summary calls unless new input supplies it. Tests must distinguish facts never retained, facts lost through rewriting, and facts present but ignored by the action model.

Stored history offers a route for repair only if the host retrieves it. Automatic retrieval is not assumed here. Retrieval, explicit constraint stores, and protected excerpts would each need separate cost and correctness tests.

VII. Independent Fixtures and Proof Obligations

These fixtures require no model service. An independent oracle enumerates boundaries from zero through b_0, tests closure against all key occurrences, and selects the greatest passing boundary. Agreement with this slow oracle checks the scan without copying its control flow.

For the plain-text fixture, use ten non-system messages named `m0` through `m9`, all with empty key sets, and choose $r = 8$. The expected boundary is 2. A stub returns one fixed nonempty summary message `s1`; the new context is `s1,m2,...,m9`. Append `m10,m11` and compact again with the same recent count. The second prefix is exactly `s1,m2,m3`, and the tail is `m4,...,m11`. This checks repeated compaction and the rule that earlier summaries enter later summary input. The stub establishes shape only, not semantic quality.

For the linked fixture, use Table 1 and $r = 2$. The expected boundary is 1, while the defined one-step repair returns 2. For an approval-only bridge, use three messages: call `c`, request for call `c` with approval `p`, and response to `p`. Choose $r =$

1. Closure returns boundary zero by following `approval:p` and then `call:c`. This input has no result yet. A separate completion check must block a model continuation that requires the local call to have finished. The case demonstrates why successful closure and complete execution must remain separate outcomes.

For edge cases, an empty non-system sequence returns unchanged; system messages remain intact. A recent count at least the message count also returns unchanged. For $r \geq 1$, a newest result linked to the first call forces boundary zero regardless of intervening text. Reuse the same literal text for a call ID and an unrelated approval ID to check namespace separation. A result without an earlier call, or an approval response without an earlier request, must fail source validation even when its key set is closed.

For commit fixtures, supply a ticket at revision 7 and a valid stub summary. If the state still has revision 7, acceptance installs the candidate at revision 8. If another writer has first appended input and moved to revision 8, acceptance must fail and preserve that append. Run the same test with changed tool schemas instead of new input. Empty output, a length-cut response, and an over-budget candidate must all leave the context and task history unchanged. The summary-call record may remain for cost accounting.

A separate trigger experiment can use the explicitly defined estimate $\hat{t} = \lceil \frac{J}{4} \rceil$, where J is the character count of the complete JSON request, and trigger at 90 percent of a declared input window. This is a baseline heuristic, not a token law. Compare it with an exact tokenizer or trusted provider counter. Keeping trigger logic separate from acceptance prevents a cheap scheduling estimate from becoming an unsupported proof of size fit.

VIII. Completed Finite Evaluation

The Python experiment implements the scan, a set-intersection oracle over every cut, and the interval oracle of Proposition 3. The recorded artifact includes source hashes, seed counts, checks, results, and limits. The domains and methods below define the inputs independently of a particular product. The experiment uses no model or provider service.

For exhaustive checking, each message is one of the four subsets of two keys. We enumerate every history of length zero through five, giving $\sum_{n=0}^{5} 4^n = 1365$ histories. For each history, we test every recent count from zero through its length. This yields 7,737 boundary cases. The scan agrees with the interval oracle in every case. The same cases check idempotence and monotonicity in recent count. This covers all key incidences in that finite domain; it does not cover all typed tool protocols.

For a wider generated domain, use seeds 0 through 29 and 40 histories per seed. Each history draws a length from 0 through 44 and a key count from 0 through 24. For a nonempty history, each key occurs at one through five distinct positions, capped by the history length. The recent count ranges from zero through the history length plus four. The source code fixes the draw order. Keys use separate call and approval prefixes, but their positions are arbitrary. Some inputs are not valid call/approval histories. They test the boundary theorem on general key sets.

TABLE II
Exact totals over the same 1,200 generated key-set histories. A split key occurs on both sides of a cut. Retained messages are summed over histories; they are not token counts.

Method	Histories with a split	Split keys	Retained messages
Recent window	727	4,591	14,682
One-step repair	92	268	23,132
Full closure	0	0	23,441

All 1,200 scan outputs also agree with the set-intersection oracle. Full closure keeps 309 more messages in total than one-step repair on these inputs. The table measures the structural cost of fixing those cuts. It does not show lower latency, lower token cost, or better answers. The fixed generator can create many long overlapping key spans, so its failure proportions should not be used as estimates for real workloads. We report exact finite counts rather than population confidence intervals. The experiment performs 24,420 checks in total, including the exhaustive, generated, and hand-written cases; checks within one history are not independent observations.

IX. Proposed Model Evaluation

A. Questions and test material

Measure split exchanges, forced retention, and semantic retention under a fixed budget. Test repeated compaction separately. Deterministic record tests establish structure; model-based task tests assess meaning.

The structural suite should generate well-formed traces with unique IDs and known ground-truth components. Vary call span, the number of interleaved calls, approval depth, multiple keys per message, recent-tail length, and the amount of unrelated text between linked records. Include empty histories, all-system histories, no removable prefix, and a chain that reaches the first message. Separate malformed-input cases should inject missing records and reused IDs. A validator must report those cases as invalid source data, rather than credit the compactor for repairing them.

The task suite should use controlled tool environments with known results and no live outside writes. Candidate tasks include joining table results, editing files under named constraints, and planning a sequence of approved read and write actions in a sandbox. Each trace needs a list of facts and constraints required by later steps. Place some early, some near the cut, and some in the retained tail. Vary whether later work asks directly for a fact or needs it to choose an action. Record tool outputs so every method sees the same evidence.

Vary token length, including request overhead, and the number of compaction rounds. Split by trace family to keep near-duplicates out of different splits. Use a separate pilot to set sample size and runtime limits, then freeze those choices before the main comparison.

B. Baselines and ablations

Compare a full-history condition where it fits, a plain recent window, a prefix summary with a fixed cut, a summary with call/result closure only, and the full call/result/approval closure method. Use the defined one-step repair to test whether transitive expansion matters. For the call-only ablation, keep call keys on calls, results, and approval requests, but remove approval keys everywhere. Approval responses then have no link keys. A text-compression baseline can act on the same older prefix, with typed records protected by the same boundary. This isolates the content compressor from structural selection.

Full history is a reference only where it fits the same budget. Report infeasible cases. Keep the action model, tool definitions, output reserve, and task seeds constant across feasible comparisons. Include summary work in latency and token use.

TABLE III
PROPOSED ABLATIONS. THEY ARE EXPERIMENTAL CHOICES, NOT COMPLETED RESULTS.

Ablation	Question isolated
Remove approval keys	Do approval links change closure or accepted input?
One-step expansion	Do newly added messages need further expansion?
Change recent count	What is the cost of the required tail?
Free text or fixed fields	Does summary form change constraint retention?
Exact or estimated count	How often does the trigger or target miss?
One or repeated rounds	How much loss comes from rewriting summaries?

C. Outcomes and analysis

Report split-key rate, missing required record rate, provider-adapter rejection rate, and closure expansion in messages and tokens. The split-key rate must use the source graph as its oracle. Provider acceptance is a different outcome: a valid closed graph can still violate provider-specific layout rules. Report no-removable-prefix cases and budget failures separately from invalid source records.

For meaning, measure exact retention of named constraints, correct use of facts in later actions, forbidden-action rate within the sandbox, and final task success. Evaluate the summary itself as well as the action that follows it. Use exact checks for IDs, numbers, and action constraints. For judgments that need people, define a rubric before testing, mask the method name, use two independent judgments, and report agreement and resolved disagreements. No such judgments have been collected for this paper.

Efficiency measures should include the actual input and output tokens of every summary and action call, compaction latency, total task latency, and peak working-context size. Report tokenizer estimation error by content type. A character-based estimate can behave differently for source code, non-English text, JSON, and media references. Keep failed and retried summary calls in the cost account.

Define two distinct comparisons before collecting outcomes. The structural comparison holds the source trace and required tail fixed and measures cut validity and retained size. The semantic comparison holds the tool environment state fixed at a cut, supplies each method's context, and measures the resulting continuation. A comparison at a fixed nominal target must also report actual retained tokens and refused compactions. Otherwise a method can look safer merely by retaining more input or refusing harder tasks. Include refusals in the all-task denominator and report feasible-task results as a second, conditional view.

Use paired traces across methods and multiple model seeds where supported. Report paired differences with confidence intervals obtained by resampling whole tasks. A task with many compactions must remain one cluster; treating each call as independent would understate uncertainty. Predeclare constraint-violation rate and task success as primary model outcomes. Treat later subgroup findings as exploratory. Publish raw counts, denominators, infeasible cases, and score distributions, so a good mean cannot hide rare structural failures.

X. Limits and Reproduction

The finite artifact checks an abstract key-set algorithm. It does not establish that a provider adapter exposes every required key, that a live tokenizer gives an exact bound, or that a database implements the snapshot and revision contract. Those claims need separate implementation tests. Explicit keys miss data dependence between unlinked calls, free-text references, and implied constraints. Whole messages can join independent calls and force a large suffix. Closure preserves those records without deciding their usefulness or making an oversized tail fit.

The summary model may omit facts, invent completion, or copy hostile text from tool output. JSON framing and disabling tools during summarization constrain the request shape but do not prove resistance to such errors. This paper does not establish a security boundary or an approval policy. Permission checks and action rules remain the host's responsibility.

A reproduction should publish the fixture generator, serialized traces, oracle outputs, summary instruction, tokenizer version, and complete provider request format. Model runs also need model version, settings, date, seeds when available, and recorded tool outputs. The boundary proof, finite evaluation, and exact fixtures can be checked without a provider. The model-based study remains a protocol: its task corpus, judgments, and model runs have not been produced.

XI. Conclusion

Context compaction for tool agents has a structural obligation before it has a compression goal. The defined backward boundary scan keeps the smallest contiguous suffix that contains a chosen recent tail and all of its keyed exchanges, under explicit assumptions. The interval oracle and completed finite tests check the boundary implementation independently.

Both the proof and the measured cases explain why a fixed sliding window and a one-step repair can fail. It also exposes a hard limit: the closed tail may consume the whole budget. A useful evaluation must then ask a separate question about the summary's meaning. Preserving tool links, fitting input, and retaining user constraints are distinct outcomes, and all three need visible evidence.

References

[1] C. Packer *et al.*, "MemGPT: Towards LLMs as Operating Systems." [Online]. Available: https://arxiv.org/abs/2310.08560v2

[2] X. Wu *et al.*, "ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization." [Online]. Available: https://arxiv.org/abs/2509.13313v3

[3] R. Ye *et al.*, "AgentFold: Long-Horizon Web Agents with Proactive Context Management." [Online]. Available: https://arxiv.org/abs/2510.24699v1

[4] Microsoft, "Compaction." Accessed: Oct. 27, 2026. [Online]. Available: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/conversations/compaction

[5] Cloudflare, "Conversation state and memory." Accessed: Oct. 27, 2026. [Online]. Available: https://developers.cloudflare.com/agents/concepts/conversation-state-and-memory/

[6] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 13358–13376. doi: 10.18653/v1/2023.emnlp-main.825.

[7] H. Jiang *et al.*, "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2024, pp. 1658–1677. doi: 10.18653/v1/2024.acl-long.91.

[8] N. F. Liu *et al.*, "Lost in the Middle: How Language Models Use Long Contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024, doi: 10.1162/tacl_a_00638.

[9] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[10] T. Schick *et al.*, "Toolformer: Language Models Can Teach Themselves to Use Tools." [Online]. Available: https://arxiv.org/abs/2302.04761

[11] C.-P. Hsieh *et al.*, "RULER: What's the Real Context Size of Your Long-Context Language Models?," in *Conference on Language Modeling*, 2024. [Online]. Available: https://arxiv.org/abs/2404.06654

[12] S. Chen, "Governance Decay: How Context Compaction Silently Erases Safety Constraints in Long-Horizon LLM Agents." [Online]. Available: https://arxiv.org/abs/2606.22528v2

[13] E. Pakhomov and E. Nijkamp, "Does an Agent's History Tell You When Compaction Will Hurt? A Modest, Bounded Effect on the TRACE Paired-Replay Corpus." [Online]. Available: https://arxiv.org/abs/2610.08722v1